

Python pour le trading — Document 6/8

L'interaction avec le monde extérieur

1. Les requêtes HTTP

Le **HTTP** est le protocole d'échange du web. Beaucoup d'API de marché s'interrogent par des requêtes HTTP : on demande une information (le dernier prix, l'historique), le serveur répond. En Python, la bibliothèque `requests` est la plus simple.

```
import requests

# GET : demander une information
reponse = requests.get("https://api.exemple.com/prix/ES")
reponse.status_code      # 200 = succès, 404 = introuvable, etc.
donnees = reponse.json() # convertir la réponse JSON en dict Python

# POST : envoyer une information (par ex. passer un ordre)
ordre = {"symbole": "ES", "quantite": 5, "prix": 5234.50}
reponse = requests.post("https://api.exemple.com/ordres", json=ordre)
```

Une requête GET **demande** une donnée ; une requête POST **envoie** une donnée (par exemple un ordre). Le `status_code` indique si la requête a réussi (200) ou échoué (404, 500...). `reponse.json()` convertit la réponse en dictionnaire Python exploitable. C'est le mécanisme de base pour les API « requête-réponse », adaptées aux données qu'on consulte ponctuellement.

***Pourquoi ça compte.** Récupérer un historique, consulter un solde, passer un ordre par API REST : autant d'opérations HTTP. Maîtriser GET/POST et la lecture du `status_code` est la base de toute connexion à une plateforme.*

2. Les WebSockets : le temps réel

Le HTTP fonctionne en « demande puis réponse », ce qui est inadapté pour un flux continu. Pour recevoir les prix **en temps réel**, on utilise un **WebSocket** : une connexion permanente où le serveur **pousse** les données dès qu'elles arrivent, sans qu'on ait à redemander.

```
import websockets
import asyncio

async def suivre_prix():
    url = "wss://flux.exemple.com/marche"
    async with websockets.connect(url) as ws:      # connexion permanente
        await ws.send({'abonnement': "ES"})        # on s'abonne à un flux
        async for message in ws:                   # on reçoit en continu
            traiter(message)                        # chaque prix dès son arrivée

asyncio.run(suivre_prix())
```

Questions : j'aimerais que tu ajoutes le meme exemple mais avec `thread`

La différence avec HTTP est fondamentale. Avec `websockets.connect`, on ouvre une connexion qui reste ouverte. On s'abonne à un flux, puis `async for message in ws` reçoit chaque nouvelle donnée dès que le serveur l'envoie — pas besoin de redemander. On reconnaît `async` et `await` du document 4 : les WebSockets vont de pair avec `asyncio`, car suivre plusieurs flux est massivement I/O-bound. C'est le mécanisme des données de marché en direct.

Pourquoi ça compte. *Reconstruire un order flow en temps réel, repose sur les WebSockets : le serveur pousse chaque tick instantanément. C'est la fondation de toute surveillance de marché en direct, au cœur du poste de trade operations.*

3. Le JSON : le format d'échange

Le **JSON** est le format texte universel pour échanger des données structurées entre systèmes. Presque toutes les API parlent JSON. Python le convertit facilement vers et depuis ses dictionnaires.

```
import json

# D'un objet Python VERS du texte JSON (sérialiser)
ordre = {"symbole": "ES", "quantite": 5, "prix": 5234.50}
texte = json.dumps(ordre)
# -> '{"symbole": "ES", "quantite": 5, "prix": 5234.5}'

# Du texte JSON VERS un objet Python (désérialiser)
recu = '{"symbole": "NQ", "quantite": 2}'
donnees = json.loads(recu)
donnees["symbole"] # -> "NQ"
```

Deux opérations symétriques : `json.dumps` transforme un objet Python en texte JSON (pour l'envoyer), `json.loads` fait l'inverse (pour lire ce qu'on reçoit). On parle de **sérialisation** et de **désérialisation**. C'est ainsi qu'un ordre devient un message transmissible, et qu'un message reçu redevient un dictionnaire exploitable. Le moyen mnémotechnique : « s » comme `dumps` pour *string* (vers le texte).

Pourquoi ça compte. *Chaque message échangé avec une plateforme — ordre envoyé, prix reçu — transite en JSON. Savoir le sérialiser et le désérialiser proprement est indispensable, et c'est aussi là que surviennent les écarts de données à résoudre (formats divergents entre sources).*

Questions : j'aimerais que tu expliques et développe sur les json, format, etc. Ex: `json.dumps` avec différents objets, idem pour `loads`. J'aimerais aussi que tu expliques la structure des json etc

4. Les bases de données

Une **base de données** stocke durablement les données de façon structurée et interrogeable. L'offre Tower mentionne le reporting et la réconciliation, qui s'appuient sur des bases. Python s'y connecte, et le langage d'interrogation standard est le **SQL**.

```
import sqlite3

conn = sqlite3.connect("trading.db")
curseur = conn.cursor()
```

```
# Interroger : récupérer les ordres sur ES
curseur.execute("SELECT symbole, prix FROM ordres WHERE symbole = ?", ("ES",))
lignes = curseur.fetchall()    # -> liste des résultats

# Insérer un nouvel ordre
curseur.execute("INSERT INTO ordres (symbole, prix) VALUES (?, ?)",
                ("NQ", 18450.0))
conn.commit()                  # valider l'écriture
conn.close()
```

Question: explique le connect(["trading.db"](#)). Le curseur, etc.

On ouvre une connexion, on envoie des requêtes SQL via un curseur. `SELECT ... WHERE` récupère des données, `INSERT` en ajoute. Détail crucial : les `?` sont des **paramètres**, et la valeur est passée séparément. C'est essentiel pour la **sécurité** (cela évite les injections SQL) et la fiabilité. `commit` valide définitivement une écriture. C'est le socle du stockage et de la réconciliation de positions.

***Pourquoi ça compte.** Le reporting PnL, l'historique des transactions et la réconciliation avec les relevés de courtiers reposent sur des bases de données. Savoir interroger et écrire en SQL, de façon sûre (paramètres), est directement lié aux responsabilités du poste.*

5. La lecture et l'écriture de fichiers

Les données circulent aussi sous forme de **fichiers** : un relevé de courtier en CSV, un historique exporté, une configuration. On lit et écrit ces fichiers en s'appuyant sur le context manager `with` du document 3, qui garantit leur fermeture.

5.1 CSV avec Pandas

```
import pandas as pd

# Lire un relevé de courtier
df = pd.read_csv("releve_courtier.csv")

# ... traitement, réconciliation ...

# Écrire le résultat
df.to_csv("reconciliation.csv", index=False)
```

Pour des données tabulaires, Pandas (document 5) lit un CSV en une ligne avec `read_csv` et l'écrit avec `to_csv`. C'est la façon la plus directe de charger un relevé de courtier pour le comparer aux positions internes — exactement le type de tâche de réconciliation du poste.

5.2 Le format Parquet pour les gros volumes

```
# Parquet : format compact et rapide pour de gros historiques
df.to_parquet("historique_es.parquet")
df = pd.read_parquet("historique_es.parquet")
```

Pour de gros historiques de marché, le format **Parquet** est préférable au CSV : il est compressé et bien plus rapide à lire et écrire, car il stocke les données par colonnes. C'est le format de choix quand les volumes deviennent importants — le cas typique des données de marché.

***Pourquoi ça compte.** Charger un relevé pour le réconcilier, sauvegarder un historique pour l'analyser : la manipulation de fichiers est quotidienne. CSV pour les échanges simples, Parquet pour les gros volumes — le bon format évite des lenteurs inutiles.*

Récapitulatif

1. **HTTP (requests)** : GET pour demander, POST pour envoyer ; vérifier le `status_code`.
2. **WebSockets** : connexion permanente où le serveur pousse les données en temps réel ; va de pair avec `asyncio`.
3. **JSON** : le format d'échange ; `dumps` pour sérialiser, `loads` pour désérialiser.
4. **Bases de données (SQL)** : SELECT/INSERT via un curseur ; utiliser des paramètres (?) pour la sécurité ; `commit` pour valider.
5. **Fichiers** : `read_csv/to_csv` pour le tabulaire, Parquet pour les gros volumes ; toujours via `with`.

Questions de vérification

Essaie de répondre avant de lire la réponse, fournie juste en dessous, en retrait.

Sur HTTP et les WebSockets

Q1. Quelle est la différence entre une requête GET et une requête POST ?

Réponse. GET demande une information au serveur ; POST envoie une information au serveur (par exemple pour passer un ordre).

Q2. Pourquoi le HTTP est-il inadapté à un flux de prix en temps réel ?

Réponse. Parce qu'il fonctionne en demande-réponse : il faudrait redemander sans cesse. Un WebSocket maintient une connexion ouverte où le serveur pousse les données dès qu'elles arrivent.

Q3. Qu'est-ce qu'un WebSocket et avec quel outil va-t-il de pair ?

Réponse. Une connexion permanente où le serveur envoie les données dès qu'elles arrivent, sans redemander. Il va de pair avec `asyncio`, car suivre des flux est de l'I/O-bound.

FAUX! Il y a `threading` aussi!!

Sur le JSON

Q4. Que font `json.dumps` et `json.loads` ?

Réponse. `dumps` sérialise : il transforme un objet Python en texte JSON (pour l'envoyer). `loads` désérialise : il transforme du texte JSON reçu en objet Python.

Q5. Pourquoi le JSON est-il important pour un système de trading ?

Réponse. Parce que chaque message échangé avec une plateforme (ordre envoyé, prix reçu) transite en JSON. C'est aussi un endroit où surviennent des écarts de données dus à des formats divergents entre sources.

Sur les bases de données

Q6. Quelle est la différence entre SELECT et INSERT en SQL ?

Réponse. SELECT récupère des données existantes ; INSERT ajoute de nouvelles données dans la table.

Q7. Pourquoi passer les valeurs via des paramètres (?) plutôt que de les insérer dans la requête ?

Réponse. Pour la sécurité (éviter les injections SQL) et la fiabilité. La valeur est passée séparément de la requête, jamais collée dans le texte.

Q8. À quoi sert commit ?

Réponse. À valider définitivement une écriture (INSERT, UPDATE) dans la base. Sans commit, la modification n'est pas enregistrée.

Sur les fichiers

Q9. Avec quoi lit-on et écrit-on un CSV en Pandas ?

Réponse. read_csv pour lire, to_csv pour écrire. C'est la façon la plus directe de charger un relevé de courtier ou de sauvegarder un résultat.

Q10. Pourquoi préférer Parquet au CSV pour de gros historiques ?

Réponse. Parquet est compressé et bien plus rapide à lire et écrire, car il stocke les données par colonnes. C'est le format de choix pour les gros volumes de données de marché.

Document suivant (7/8) : la qualité, le test et l'outillage — pytest, mocking, logging, débogage et profilage de performance, ce qui distingue du code personnel d'un code de production.